4 Chapter 4: Delay

4.3 RC Delay Model

The RC model uses a switch in series with a resistor to model a MOS transistor, with effective resistance R. The equivalent resistances and capacitances will naturally affect the delay parameters of the given network.

Using the RC model, estimate the delay of a fanout-of-1 inverter (two inverters in series). Recall that the width of the pMOS will be twice that of the nMOS transistor. Using the RC model, we can approximate the fanout-of-1 inverter's delay by modeling the nMOS and pMOS transisors has having the same resistance R, but the pMOS will have twice the capacitance of the nMOS, $2C$. Diffusion capacitance exists in both nMOS and pMOS transistors. At any given state for the inverter, account for which capacitors are shorted out; I think this implies that an inverter would have a different delay based on whether it's pulling up or down?

Compared the Shockley model (ideal), SPICE (simulated), and the simplified RC model, where the RC model ends up being more conservative.

Moving on to a three-input NAND gate, if we want to achieve rise and fall resistances equal to a unit inverter, the nMOS transistors' (in series) widths will need to be three times wider than normal to achieve a resistance of $\frac{1}{3}R$ for each one, summing to a total series resistance of R. In the pull-up network, the widths stay at their normal value (two times that of a "normal" nMOS) since these transistors are in parallel. The widened nMOS transistors will now carry a capacitance of $3C$ each, since they are 3 times wider.

Practice showing RC-equivalent circuits for different standard logic gates (OR-AND/AND-OR invert compound gates?)

Elmore delay simplifies delay into a resistor ladder, where

$$t_{pd} = \sum_{\text{nodes} i} R_{is} C_i.$$

Fanout is a term that describes how many *other* gates are being driven by the gate being analyzed, a condition which will affect the delay parameters.

Using Elmore delay, we estimate the worst-case rising and falling delay of a 3-input NAND gate driving h identical gates. For rising delay, we construct an RC-equivalent circuit. **Review what an Elmore ladder is, derivation of Elmore delay approximation, etc**. *Side note, capacitance means that all the "capacitors" will need to discharge before you can pull something down to ground.* Then, using the Elmore delay for the rise time,

$$t_{pdr} = (9 + 5h)...continue$$

Get familiar with using Elmore delay approximations to find the delay of a gate given fanout of h identical gates (or other types of gates?) Two components of delay – parasitic delay and effort delay (define these). Depending on the input, best-case (contamination delay) could be substantially less than the propagation delay (goes back to parallel resistances in the pull-up network of a three-input NAND).

Research timing arc a little better. Delay varies with direction and input, lots of interesting details here to dig into. How many rows would a timing arc table need based on the effect of inputs/outputs on delay? (Try this with multiple different types of gates). Capacitance is also dependent on layout. Shared diffusion nodes reduce total capacitance. (Practice estimating shared diffusion effect on capacitance). Stick diagrams are even harder to think about now, awesome. Means layout will affect delay.

Not going to worry about DC transfer characteristics much from here on out. Instead of using Shockley model, we're simplifying nMOS and pMOS transistors into an RC-equivalent model. Then, we can use elementary circuit analysis techniques to estimate the delay based on the network's RC characteristics.

Transistor width becomes very important when dealing with delay, especially considering the substantial

difference between charge carrier mobility in pMOS versus nMOS devices. Remember that in many networks, only some switches are on at any given input, so certain RC characteristics are considered or ignored based on the input to the network. Distinguish gate and diffusion capacitance (review what causes these capacitive effects). Elmore delay models the approximate delay of a network based on its fanout (how many other gates it's driving).

Example: Estimate worst-case rising and falling delay of a 3-input NAND driving h identical gates. In a NAND gate, we look at gate (parasitic) and diffusion (fanout) capacitance. In rising delay, we don't care about the pull-down network, since rising delay implies the pull-up network is active and under consideration (will some nodes lag behind others in a parallel network? a series network? why?) For falling delay, we consider the time it takes for the diffusion capacitance of the three gates to discharge (not simultaneously, but in succession) and the gate capacitance $(9 + 5h)$C. Rise and fall time will be different. There is an intrinsic built-in capacitance no matter what h equals, and then added capacitance based on fanout.

Parasitic delay vs effort delay. Parasitic delay is independent of load, whereas effort delay is based on fanout. Best-case (minimum, contamination) delay can be substantially less than worst-case (maximum, propagation) delay. Useful to estimate the best-case worst-case range? Diffusion capacitance assumptions are based on layout. Layout can reduce delay below worst-case assumptions.

4.4 Linear Delay Model

Logical effort is based primarily on the

Sizing gates and designing gate networks is automated, so we're learning about the mathematical foundations of these systems by learning logical effort theory. Logic networks can be immensely complicated, and we'd like to make layout as efficient as possible mathematically. Example of the design of a decoder for a register file. 16-word register file, each word is 32 bits wide. Constraints: max fanout of 10. Each bit presents load of 3 unit-sized transistors.

Begin by expressing delay as process-independent (sizing) as $d = \frac{d_{abs}}{\tau}$. Delay has the two components, effort f and logical g, $d = f + p$. $f = gh$, where $h = \frac{C_{out}}{C_in}$. None of this makes sense but hopefully it will later. Logical effort is defined as the ratio of input capacitance of a gate to the input capacitance of an inverter delivering the same output current. For a unit inverter, the logical effort $g = \frac{3}{3} = 1$, since $C_{in} = 3$. If, for example, the input capacitance of a gate was $C_{in} = 4$, then $g = \frac{4}{3}$. You can measure this from delay vs fanout plots (NAND has a lower logical effort than NOR, and is preferred by logic designers for this reason). Parasitic delay and logical effort for common gates are good values to memorize probably? Delay plots show normalized delay d as a function of incremental electrical effort $h = \frac{C_{out}}{C_{in}}$.

What about NOR2? Upcoming homework problem.

Example: Ring oscillator. N-stage ring oscillator, estimate the frequency oscillation. Logical effort of each inverter is $g = 1$, electrical effort is $h = 1$, parasitic delay is $p = 1$, stage delay $d = 1 + 1 = 2$, and frequency $f_{osc} = \frac{1}{2Nd} = \frac{1}{4N}$.

FO4 inverters are used as a standard delay, and is one unit inverter driving four other inverters. Given values, $g = 1$, $h = 4$, $p = 1$, $d = 5$ (memorize this).

Logical effort generalizes to multistage networks, with path logical effort $G = \Pi g_i$, etc (check slides for this). What about branching paths? This is confusing as hell, I need to review it 700 times. This leads to the concept of branching effort. Delay is smallest when each stage bears same effort, and minimum delay of N stage path is $D = NF^{\frac{1}{N}} + P$. (Apparently this helps design fast circuits). Practice selecting gate sizes for least delay, given a logic network. Fanout/branching are sort of synonymous. Learn to convert delay to units of FO4.

Paradoxically, more gates can sometimes reduce delay by using intermediate "amplifier" stages to drive

more gates more efficiently (increasing transistor widths). Optimal number of stages is a consideration (non-integer inverter widths?) Mathematically optimal solution doesn't always match the parts that are avilable to us. Compare alternatives with a spreadsheet listing options + N,G,P,D values.

Verilog register files are basically doing all this math under the hood.

Logical effort has limitations. For example, we need the path to comput G, but don't know the number of stages without G. The delay model also neglects input rise time effects, interconnect effects (wire resistance and capacitance that increases with reductions in size), and does not account for constrained delay.

Xilinx delay reports? Timing analysis field.

In summary, despite limitations, NANDs are faster than NORs, FO4 is the standard, fewer stages don't necessarily mean less delay, and it provides language for discussing fast circuits.

Cell catalog for a TSMC 0.18µm. Looking at an AOI121. What information is provided? Comes in several sizes (physical size). Height is usually the same so we can align cells to VDD and GND, width changes based on transistor sizing/gate type (I assume). Power consumption is listed, and wider transistors consume more power. Capacitance is also proportional to transistor size, and delay is given in nanoseconds, while load adustment is given in $\frac{\text{ns}}{\text{pF}}$, based on the amount of input capacitance to the next stage.

Eddicient Verilog code, synthesis tools.